ТЕХНИЧЕСКОЕ ЗАДАНИЕ ДЛЯ CLAUDE CODE

Лендинг + приём заявок + админка

Проект 1 из портфолио. White-label, компонентный подход, тестовая среда, деплой на VDS.

Бэкенд	Python 3.14 · FastAPI · SQLAlchemy 2 (async) · Alembic
База данных	PostgreSQL 18
Фронтенд	Vue 3 · TypeScript · Vite · vite-ssg · своя дизайн-система
Инфраструктура	Docker Compose · Caddy (HTTPS) · VDS Beget
Тесты	pytest + testcontainers · Vitest · Storybook · Playwright · Lighthouse CI
Версия документа	1.0 · 20.10.2026

Содержание

0. Как работать с этим документом ... 3
1. Цель и рамки ... 4
1.1. Что строим ... 4
1.2. Честность демо (MUST) ... 4
1.3. В рамках MVP ... 4
1.4. Вне рамок MVP ... 5
2. Стек ... 5
3. Архитектура ... 6
3.1. Сервисы ... 6
3.2. Поток заявки ... 6
3.3. Слои бэкенда ... 7
4. Структура монорепозитория ... 7
5. Конфигурация и окружения ... 8
5.1. Окружения ... 8
5.2. Переменные окружения API ... 8
6. База данных ... 10
6.1. Общие правила ... 10
6.2. Таблицы ... 10
7. Бэкенд ... 12
7.1. Приложение ... 12
7.2. Сессия БД ... 13
7.3. Формат ошибок — RFC 9457 (application/problem+json) ... 13
7.4. Публичный приём заявки ... 13
7.5. IP-адрес ... 14
7.6. Rate limit на PostgreSQL ... 14
7.7. Аутентификация админки ... 14
7.8. Логирование ... 15
7.9. Health ... 15
8. Контракт API ... 15
8.1. Публичные ... 15
8.2. Админка (cookie-сессия; для изменяющих — X-CSRF-Token) ... 16
9. Уведомления ... 17
9.1. Каналы ... 17
9.2. Воркер ... 17
10. Фронтенд: дизайн-система (packages/ui) ... 18
10.1. Принципы ... 18
10.2. Токены ... 18
10.3. Каталог компонентов ... 19
11. i18n и компонент Text ... 23
11.1. Словарь ... 23
11.2. Типизированные ключи ... 23
11.3. Компонент Text ... 24
11.4. Проверки (MUST) ... 25
12. Публичный сайт (apps/site) ... 26
12.1. Конфиг контента ... 26

12.2. Структура главной ... 26
12.3. Форма заявки — полная спецификация ... 27
12.4. Страницы ... 28
12.5. SEO ... 28
12.6. Производительность (MUST, проверяется Lighthouse CI на мобильном профиле) ... 28
13. Админка (apps/admin) ... 29
13.1. Общее ... 29
13.2. Экран «Вход» (/login) ... 29
13.3. Каркас AppShell ... 29
13.4. Экран «Заявки» (/leads) ... 30
13.5. Карточка заявки (/leads/:id, в Drawer) ... 30
13.6. Экран «Аккаунт» (/account) ... 31
14. Персональные данные (152-ФЗ) ... 31
15. Безопасность — чек-лист ... 32
16. Доступность ... 32
17. Тестовая среда ... 33
17.1. Пирамида ... 33
17.2. Бэкенд: фикстуры ... 33
17.3. Фронтенд ... 34
17.4. E2E (Playwright) ... 34
17.5. Команды ... 35
18. Качество кода ... 35
19. Docker и инфраструктура ... 36
19.1. Образы ... 36
19.2. compose.prod.yml (суть) ... 36
19.3. Caddyfile ... 37
19.4. Бэкапы ... 38
20. Деплой на VDS Beget ... 38
21. CI (GitHub Actions) ... 39
22. План реализации ... 39
23. Чек-лист приёмки ... 40
24. Переиспользование в проектах 2 и 3 ... 40
25. Типичные ошибки, которых нельзя допускать ... 40

0. Как работать с этим документом

Это техническое задание для **Claude Code**. Документ самодостаточен: по нему проект собирается с нуля до работающего продакшена на VDS.

Правила для исполнителя (Claude Code):

1. Работай строго по этапам из раздела 22. Не начинай следующий этап, пока у текущего не выполнен Definition of Done (DoD).
2. После каждого этапа запускай `make lint typecheck test`. Красные проверки — этап не закрыт.
3. **Версии не выдумывай.** Таблица в разделе 2 задаёт мажорные версии, проверенные на октябрь 2026. Конкретные патч-версии ставь последние стабильные на момент установки и фиксируй в lock-файлах (`uv.lock`, `pnpm-lock.yaml`).

4. Никаких заглушек `TODO`, `pass`, `...` в коде, который считается готовым. Если что-то вне рамок MVP — это фиксируется в разделе «Вне рамок», а не оставляется полуготовым.
5. **Ни одной строки текста для пользователя прямо в шаблоне.** Все тексты — через i18n и компонент `Text` (раздел 11). Это проверяется линтером.
6. Источник истины для валидации — бэкенд. Фронтенд дублирует правила для UX, но сервер проверяет всё сам.
7. Тесты бэкенда — только на настоящем PostgreSQL той же мажорной версии (testcontainers). SQLite запрещён.
8. Спрашивай у владельца только то, что нельзя знать заранее: домен, SMTP-доступы, тексты и данные бизнеса. Всё остальное решено в этом документе.

Обозначения: `MUST` — обязательно; `SHOULD` — желательно, отступление только с обоснованием в README; `MAY` — по желанию.

1. Цель и рамки

1.1. Что строим

Демо-проект для портфолио: **сайт-лендинг локального бизнеса с приёмом заявок и админкой**.

- Посетитель открывает сайт, видит услуги и цены, оставляет заявку через форму.
- Заявка сохраняется в базу, владельцу уходит уведомление (email; опционально бот в MAX).
- Владелец заходит в админку, видит список заявок, фильтрует, меняет статус, пишет заметки, выгружает CSV.

Проект **white-label**: весь контент и бренд задаются одним конфигом. Это позволяет показывать один код под разные ниши (автосервис, салон, клининг) и переиспользовать его в проектах 2 и 3 (раздел 24).

1.2. Честность демо (`MUST`)

- Бренд в конфиге по умолчанию — **вымышленный**, в подвале сайта показывается плашка «Демо-проект» (включается флагом `demo: true` в конфиге).
- **Секции «Отзывы» в демо нет.** Выдуманные отзывы не публикуются. Секция может быть добавлена только когда есть реальные отзывы реального бизнеса.
- Реквизиты, адрес, телефон в демо — явно тестовые (например `+7 (000) 000-00-00`).

1.3. В рамках MVP

- Публичный сайт: главная (секции из раздела 12), страницы «Политика обработки ПДн», «Согласие на обработку ПДн», 404.
- Форма заявки со всеми состояниями.
- Админка: вход, список заявок, карточка заявки, смена статуса, заметки, история, экспорт CSV, смена пароля, выход со всех устройств.
- Уведомления: email через SMTP; адаптер MAX за флагом.
- Тестовая среда, CI, Docker, деплой на VDS, бэкапы.

1.4. Вне рамок MVP

Редактирование контента сайта из админки, несколько ролей пользователей, мультиязычность (структура i18n готова, но язык один — русский), онлайн-оплата, мобильное приложение.

2. Стек

Мажорные версии проверены на **20.10.2026**. Патч-версии — последние стабильные при установке, фиксируются в lock-файлах.

Слой	Технология	Версия	Зачем
Язык бэкенда	Python	3.14	Актуальная стабильная ветка
Менеджер пакетов Python	uv	последняя	Быстрый, lock-файл, управление Python
Веб-фреймворк	FastAPI	≥ 0.136	Типизация, OpenAPI из коробки
Валидация / настройки	Pydantic v2, pydantic-settings	2.x	Схемы и конфиг из env
ORM	SQLAlchemy	2.x (async)	Типизированные модели, async
Драйвер БД	asyncpg	последняя	Быстрый async-драйвер
Миграции	Alembic	последняя	Версионирование схемы
БД	PostgreSQL	18	Текущая стабильная мажорная; встроенный `uuidv7()`
Хеширование паролей	argon2-cffi	последняя	Argon2id
Телефоны	phonenumbers	последняя	Нормализация в E.164
Почта	aiosmtplib + Jinja2	последние	Асинхронная отправка, шаблоны писем
Логи	structlog	последняя	JSON-логи с request_id
Node.js	Node.js LTS	24 (Active LTS)	26 станет LTS 28.10.2026 — можно перейти после этой даты
Менеджер пакетов JS	pnpm	10.x	Workspaces, строгие зависимости
Фреймворк UI	Vue	3.5+	Composition API, `<script setup>`
Сборка	Vite + @vitejs/plugin-vue	8.x / 6.x	Сборка обоих фронтендов
Типизация	TypeScript + vue-tsc	5.x+ / 3.x	Строгая проверка `.vue`
Пререндер сайта	vite-ssg	последняя	SEO: сайт отдаётся готовым HTML
Роутинг / состояние	Vue Router, Pinia	4.x / 3.x	Админка
Серверное состояние	@tanstack/vue-query	5.x	Кэш, повторы, инвалидация запросов в админке
i18n	vue-i18n	11.x	Тексты, плюрализация, интерполяция
Валидация на клиенте	zod	4.x	Схемы форм
API-клиент	openapi-typescript + openapi-fetch	последние	Типы и клиент генерируются из OpenAPI бэкенда

Слой	Технология	Версия	Зачем
Веб-сервер / TLS	Caddy	2.x	Автоматический HTTPS, reverse proxy
Контейнеры	Docker Engine + Compose v2	последние	Одинаковое окружение dev/test/prod

Совместимость `vite-ssg` (`MUST` проверить на этапе M7): если на момент установки `vite-ssg` не поддерживает установленную мажорную версию Vite, Vite фиксируется на последней мажорной, которую `vite-ssg` поддерживает. Причина записывается в README.

Почему не Telegram: в России Telegram работает с перебоями с февраля 2026; уведомления и мини-приложения делаем под MAX и email.

Почему без Tailwind: проект строится на собственной дизайн-системе с токенами. Приложения используют только компоненты из `@app/ui`, утилитарные классы в коде приложений не нужны.

Почему без Redis: для масштаба одного бизнеса rate limit и очередь уведомлений надёжно работают на PostgreSQL (разделы 7.6 и 9). Меньше сервисов — меньше точек отказа на маленькой VDS.

3. Архитектура

3.1. Сервисы

```
                Интернет
                   |
              [ Caddy :443 ]  -- автоматический HTTPS (Let's Encrypt)
              /            \
example.ru (сайт)      admin.example.ru (админка)
  |   статический HTML     |   SPA
  |   /api/* -> api        |   /api/* -> api
               \         /
              [ api: FastAPI ]  (uvicorn, 2 воркера)
                   |
              [ PostgreSQL 18 ]
                   |
              [ worker ]  -- читает outbox, шлёт email / MAX
              [ backup ]  -- ежедневный pg_dump
```

- **Одинаковый origin для API.** И сайт, и админка ходят в API по относительному пути `/api/...`. Caddy проксирует `/api/*` на сервис `api`. CORS не нужен, cookie админки не уходят на чужой домен.
- **Админка на отдельном поддомене.** Cookie сессии привязаны к `admin.example.ru` и никогда не отправляются с публичного сайта.

3.2. Поток заявки

1. Посетитель отправляет форму → `POST /api/v1/public/leads`.
2. API проверяет: rate limit → honeypot и время заполнения → валидация → нормализация телефона.
3. **В одной транзакции:** запись согласия (`consents`) → заявка (`leads`) → событие `created` (`lead_events`) → задачи уведомлений (`notification_outbox`).
4. Ответ `201` с номером заявки. Отправка уведомления **не блокирует** ответ.

5. Воркер забирает задачу из outbox, отправляет письмо, ставит статус `sent`; при ошибке — повтор с экспоненциальной задержкой.

Почему outbox: если SMTP недоступен, заявка всё равно сохранена, а уведомление уйдёт позже. Заявка и задача на уведомление пишутся атомарно — не бывает «заявка есть, уведомления нет» или наоборот.

3.3. Слои бэкенда

```
routes (HTTP)  ->  services (бизнес-логика)  ->  repositories (SQL)  ->  models
     |                   |
  schemas (Pydantic: вход/выход)     domain (enum, правила, ошибки домена)
```

- `routes` — только HTTP: принять, вызвать сервис, вернуть схему. Без SQL и бизнес-правил.
- `services` — бизнес-логика и транзакции. Не знают про HTTP.
- `repositories` — весь SQL. Возвращают модели или DTO.
- Ошибки домена (`LeadNotFound`, `VersionConflict`, `RateLimited`) мапятся в HTTP в одном месте (раздел 7.3).

4. Структура монорепозитория

```
project/
├─ apps/
│  ├─ api/                         # FastAPI
│  │  ├─ app/
│  │  │  ├─ main.py               # создание приложения, middleware, роутеры
│  │  │  ├─ settings.py           # pydantic-settings
│  │  │  ├─ db.py                 # engine, sessionmaker, get_session
│  │  │  ├─ logging.py            # structlog
│  │  │  ├─ errors.py             # problem+json, маппинг ошибок
│  │  │  ├─ security/             # passwords.py, sessions.py, csrf.py, ip.py
│  │  │  ├─ domain/               # enums.py, errors.py
│  │  │  ├─ models/               # SQLAlchemy-модели
│  │  │  ├─ schemas/              # Pydantic-схемы
│  │  │  ├─ repositories/
│  │  │  ├─ services/             # leads.py, auth.py, notifications.py, ratelimit.py
│  │  │  ├─ routes/               # public_leads.py, admin_auth.py, admin_leads.py, health.py
│  │  │  ├─ notifications/        # channels/email.py, channels/max.py, channels/log.py, templates/
│  │  │  ├─ worker.py             # цикл обработки outbox
│  │  │  └─ cli.py                # create-admin, anonymize-expired
│  │  ├─ migrations/              # Alembic
│  │  ├─ tests/                   # unit/, integration/, contract/
│  │  ├─ pyproject.toml
│  │  └─ Dockerfile
│  ├─ site/                        # публичный сайт (Vue + vite-ssg)
│  │  ├─ src/
│  │  │  ├─ sections/             # Hero.vue, Services.vue, ...
│  │  │  ├─ features/lead-form/   # LeadForm.vue, useLeadForm.ts, schema.ts
│  │  │  ├─ pages/                # Home.vue, Privacy.vue, Consent.vue, NotFound.vue
│  │  │  ├─ seo/                  # head.ts, jsonld.ts, sitemap.ts
│  │  │  └─ main.ts
│  │  ├─ site.config.ts           # бренд и контент (white-label)
│  │  └─ Dockerfile
│  └─ admin/                       # админка (Vue SPA)
│     ├─ src/
│     │  ├─ layouts/AppShell.vue
│     │  ├─ pages/                # LoginPage, LeadsPage, LeadDetail, AccountPage
│     │  ├─ features/             # leads/, auth/, account/
│     │  ├─ router.ts
│     │  └─ main.ts
│     └─ Dockerfile
├─ packages/
```

```
│   ├─ ui/                          # дизайн-система: токены + компоненты + сторибук
│   ├─ i18n/                        # ru.json, типы ключей, T()
│   ├─ api-client/                  # сгенерированные типы + обёртка openapi-fetch
│   └─ config/                      # общие tsconfig, eslint, prettier, stylelint
├─ infra/
│   ├─ caddy/Caddyfile
│   ├─ compose.dev.yml
│   ├─ compose.test.yml
│   ├─ compose.prod.yml
│   └─ backup/                      # скрипт pg_dump с ротацией
├─ e2e/                             # Playwright
├─ .github/workflows/ci.yml
├─ Makefile
├─ lefthook.yml
├─ pnpm-workspace.yaml
└─ README.md
```

5. Конфигурация и окружения

5.1. Окружения

Окружение	Где	БД	Почта	Назначение
`dev`	локально, `compose.dev.yml`	PostgreSQL в контейнере	Mailpit (ловушка писем, веб-интерфейс :8025)	Разработка с hot reload
`test`	CI и локально, `compose.test.yml`	отдельный PostgreSQL	Mailpit	Интеграционные и e2e-тесты
`prod`	VDS, `compose.prod.yml`	PostgreSQL с томом	реальный SMTP	Продакшен

5.2. Переменные окружения API

Все переменные читаются через `pydantic-settings`. Секреты **никогда** не коммитятся; в репозитории только `.env.example`.

Переменная	Пример	Обязательна	Описание
`APP_ENV`	`prod`	да	`dev` / `test` / `prod`
`DATABASE_URL`	`postgresql+asyncpg://app:***@db:5432/app`	да	Строка подключения
`SECRET_KEY`	64 случайных байта в base64	да	HMAC для IP-хешей и токенов
`SITE_ORIGIN`	`https://example.ru`	да	Для ссылок в письмах
`ADMIN_ORIGIN`	`https://admin.example.ru`	да	Для ссылок в письмах
`SMTP_HOST` / `SMTP_PORT`	`smtp.example.ru` / `465`	prod	SMTP
`SMTP_USER` / `SMTP_PASSWORD`	—	prod	SMTP-учётка
`SMTP_FROM`	`Заявки <noreply@example.ru>`	prod	Отправитель
`NOTIFY_EMAILS`	`owner@example.ru`	да	Кому слать уведомления, через запятую
`MAX_BOT_TOKEN`	—	нет	Включает канал MAX
`MAX_CHAT_ID`	—	нет	Чат владельца в MAX

Переменная	Пример	Обязательна	Описание
`SESSION_TTL_HOURS`	`168`	нет	Срок сессии админки (по умолчанию 7 дней)
`LEAD_RETENTION_DAYS`	`365`	нет	Через сколько обезличивать заявки (раздел 14)
`PRIVACY_POLICY_VERSION`	`2026-10-20`	да	Версия политики, фиксируется в согласии
`RATE_LIMIT_LEADS`	`5/600`	нет	5 заявок за 600 с с одного IP
`RATE_LIMIT_LOGIN`	`10/900`	нет	10 попыток входа за 900 с на IP+email
`SMARTCAPTCHA_SERVER_KEY`	—	нет	Включает Yandex SmartCaptcha
`LOG_LEVEL`	`INFO`	нет	Уровень логов

`settings.py` (образец):

```python
from functools import lru_cache
from typing import Literal

from pydantic import PostgresDsn, SecretStr, field_validator
from pydantic_settings import BaseSettings, SettingsConfigDict


class Settings(BaseSettings):
    model_config = SettingsConfigDict(env_file=".env", extra="ignore")

    app_env: Literal["dev", "test", "prod"]
    database_url: PostgresDsn
    secret_key: SecretStr
    site_origin: str
    admin_origin: str
    notify_emails: list[str]
    privacy_policy_version: str
    session_ttl_hours: int = 168
    lead_retention_days: int = 365
    rate_limit_leads: str = "5/600"
    rate_limit_login: str = "10/900"
    smtp_host: str | None = None
    smtp_port: int = 465
    smtp_user: str | None = None
    smtp_password: SecretStr | None = None
    smtp_from: str | None = None
    max_bot_token: SecretStr | None = None
    max_chat_id: str | None = None
    smartcaptcha_server_key: SecretStr | None = None
    log_level: str = "INFO"

    @field_validator("notify_emails", mode="before")
    @classmethod
    def split_emails(cls, v: str | list[str]) -> list[str]:
        return [e.strip() for e in v.split(",")] if isinstance(v, str) else v

    @property
    def is_prod(self) -> bool:
        return self.app_env == "prod"


@lru_cache
def get_settings() -> Settings:
    return Settings()  # type: ignore[call-arg]
```

Проверка при старте (`MUST`): в `prod` приложение не стартует, если не заданы SMTP-переменные или `SECRET_KEY` короче 32 байт.

6. База данных

6.1. Общие правила

- Первичные ключи — `uuid` со значением по умолчанию `uuidv7()` (встроено в PostgreSQL 18; упорядочено по времени, хорошо индексируется).
- Все даты — `timestamptz`, хранятся в UTC. Отображение во фронтенде — в часовом поясе бизнеса из конфига.
- Расширения: `citext` (регистронезависимый email).
- Каждое изменение схемы — отдельная миграция Alembic. Миграции **не редактируются** после применения в prod.
- Имена: таблицы во множественном числе, `snake_case`, явные имена ограничений (`ck_leads_name_len`, `ix_leads_status_created_at`).

6.2. Таблицы

`admin_users`

Поле	Тип	Ограничения
`id`	uuid	PK, default `uuidv7()`
`email`	citext	NOT NULL, UNIQUE
`password_hash`	text	NOT NULL (Argon2id)
`is_active`	boolean	NOT NULL, default true
`created_at`	timestamptz	NOT NULL, default now()
`last_login_at`	timestamptz	NULL

`admin_sessions`

Поле	Тип	Ограничения
`id`	uuid	PK
`user_id`	uuid	FK → admin_users ON DELETE CASCADE
`token_hash`	bytea	NOT NULL, UNIQUE — SHA-256 от токена; сам токен не хранится
`csrf_token_hash`	bytea	NOT NULL
`created_at` / `last_seen_at`	timestamptz	NOT NULL
`expires_at`	timestamptz	NOT NULL, индекс
`revoked_at`	timestamptz	NULL
`ip_hash`	bytea	NOT NULL — HMAC-SHA256(IP, SECRET_KEY)
`user_agent`	text	NOT NULL, обрезается до 512 символов

`consents` — факт согласия на обработку ПДн

Поле	Тип	Ограничения
`id`	uuid	PK
`policy_version`	text	NOT NULL
`consent_text_sha256`	bytea	NOT NULL — хеш текста согласия, показанного человеку
`ip_hash`	bytea	NOT NULL
`user_agent`	text	NOT NULL
`given_at`	timestamptz	NOT NULL, default now()

`leads`

Поле	Тип	Ограничения
`id`	uuid	PK
`public_number`	bigint	GENERATED ALWAYS AS IDENTITY, UNIQUE — «Заявка № 123»
`name`	text	NOT NULL, CHECK длина 2–100
`phone_e164`	text	NOT NULL, CHECK `~ '^\+[1-9][0-9]{7,14}$'`
`email`	citext	NULL
`message`	text	NULL, CHECK длина ≤ 2000
`service_slug`	text	NULL — slug услуги из конфига сайта
`source`	text	NOT NULL, default `'site'`, CHECK in (`site`, `max`)
`utm`	jsonb	NOT NULL, default `'{}'` — только ключи `utm_source/medium/campaign/term/content`
`status`	lead_status	NOT NULL, default `'new'`
`version`	integer	NOT NULL, default 1 — оптимистичная блокировка
`consent_id`	uuid	FK → consents, NOT NULL
`anonymized_at`	timestamptz	NULL
`created_at` / `updated_at`	timestamptz	NOT NULL

Тип `lead_status`: `new`, `in_progress`, `done`, `spam`, `archived`.

Индексы: `ix_leads_status_created_at (status, created_at DESC)`, `ix_leads_created_at (created_at DESC)`.

`lead_events` — история и заметки

Поле	Тип	Ограничения
`id`	uuid	PK
`lead_id`	uuid	FK → leads ON DELETE CASCADE, индекс
`actor_user_id`	uuid	FK → admin_users, NULL (NULL = система)
`type`	text	CHECK in (`created`, `status_changed`, `note_added`)
`payload`	jsonb	NOT NULL — `{from, to}` или `{text}`
`created_at`	timestamptz	NOT NULL

`notification_outbox`

Поле	Тип	Ограничения
`id`	uuid	PK
`kind`	text	NOT NULL — `lead_created`
`channel`	text	CHECK in (`email` , `max` , `log`)
`payload`	jsonb	NOT NULL — `{lead_id}` ; персональные данные в payload не кладутся, воркер читает их из `leads`
`status`	text	CHECK in (`pending` , `sent` , `failed`), default `pending`
`attempts`	integer	NOT NULL, default 0
`next_attempt_at`	timestamptz	NOT NULL, default now()
`last_error`	text	NULL
`created_at` / `sent_at`	timestamptz	—

Частичный индекс: `ix_outbox_pending (next_attempt_at) WHERE status = 'pending'` .

`rate_limits`

Поле	Тип	Ограничения
`bucket`	text	Например `leads:<ip_hash_hex>`
`window_start`	timestamptz	—
`hits`	integer	NOT NULL

PK `(bucket, window_start)` . Старые окна чистит воркер раз в час.

7. Бэкенд

7.1. Приложение

- Фабрика `create_app()` в `main.py` ; жизненный цикл через `lifespan` (создание и закрытие пула БД).
- Все роутеры под префиксом `/api/v1` .
- Middleware по порядку: `RequestIdMiddleware` (берёт `X-Request-ID` или генерирует, кладёт в structlog-контекст и в ответ) → логирование запроса (метод, путь, статус, длительность; **без тела и без персональных данных**) → обработчики ошибок.
- OpenAPI доступен только при `APP_ENV != prod` (`docs_url=None, openapi_url=None` в prod). Схема для генерации клиента выгружается командой `make openapi` в `packages/api-client/openapi.json` .
- Запуск: `uvicorn app.main:create_app --factory --workers 2 --proxy-headers` . По умолчанию uvicorn доверяет `X-Forwarded-For` только от `127.0.0.1` , а Caddy стучится из другого контейнера — поэтому в compose задаётся переменная `FORWARDED_ALLOW_IPS` с адресом Caddy (или подсетью внутренней docker-сети). Значение `*` допустимо **только** если порт API не опубликован наружу и до него может достучаться лишь Caddy. Без этой настройки rate limit будет считать всех посетителей одним IP.

7.2. Сессия БД

```python
async def get_session() -> AsyncIterator[AsyncSession]:
    async with SessionLocal() as session:
        yield session
```

Транзакции открывает **сервис** (`async with session.begin():`), а не роут и не репозиторий.

7.3. Формат ошибок — RFC 9457 (`application/problem+json`)

Все ошибки API имеют один формат:

```json
{
  "type": "https://example.ru/problems/validation",
  "title": "Validation failed",
  "status": 422,
  "code": "validation_error",
  "detail": "Одно или несколько полей заполнены неверно",
  "errors": [
    {"field": "phone", "code": "phone_invalid"}
  ],
  "request_id": "01JB..."
}
```

- `code` — машинный код. **Фронтенд показывает текст по коду через i18n**, а не текст из `detail`.
- Таблица кодов:

HTTP	code	Когда
400	`bad_request`	Некорректный JSON
401	`unauthenticated`	Нет сессии или истекла
403	`csrf_failed`	Нет или неверный CSRF-токен
404	`not_found`	Ресурса нет
409	`version_conflict`	Заявку уже изменили (оптимистичная блокировка)
422	`validation_error`	Ошибки полей, массив `errors`
429	`rate_limited`	Превышен лимит; заголовок `Retry-After` в секундах
500	`internal_error`	Всё остальное; детали только в логах

- Коды ошибок полей: `required`, `too_short`, `too_long`, `invalid_format`, `phone_invalid`, `email_invalid`, `consent_required`, `unknown_service`.
- Стек-трейс никогда не попадает в ответ.

7.4. Публичный приём заявки

Схема входа:

```python
class LeadCreate(BaseModel):
    model_config = ConfigDict(extra="forbid", str_strip_whitespace=True)

    name: Annotated[str, StringConstraints(min_length=2, max_length=100)]
    phone: Annotated[str, StringConstraints(max_length=32)]
    email: EmailStr | None = None
    message: Annotated[str, StringConstraints(max_length=2000)] | None = None
    service_slug: Annotated[str, StringConstraints(max_length=64)] | None = None
```

```
    consent: Literal[True]
    policy_version: str
    utm: dict[UtmKey, Annotated[str, StringConstraints(max_length=200)]] = {}
    website: str = ""          # honeypot: должно быть пустым
    elapsed_ms: int = Field(ge=0)  # сколько форма была открыта
    captcha_token: str | None = None
```

Порядок проверок в `LeadService.create()`:

1. Rate limit по `ip_hash` (раздел 7.6). Превышен → `429`.
2. Honeypot: если `website` не пустое **или** `elapsed_ms < 3000` → ответить **`201` с фиктивным номером**, ничего не сохранять, записать в лог `spam_suspected`. Бот не должен понять, что его отсекли.
3. Если включена SmartCaptcha — проверить токен на сервере.
4. `policy_version` должна совпадать с текущей → иначе `422` (`field: consent`, `code: consent_required`) — это значит, что человек видел старую версию политики.
5. `phone` → `phonenumbers.parse(phone, "RU")`, `is_valid_number` → формат E.164. Ошибка → `phone_invalid`.
6. `service_slug` должен быть в списке разрешённых (берётся из того же конфига, что и сайт, — см. 12.1; бэкенд читает `services.json`, который генерируется при сборке сайта). Иначе `unknown_service`.
7. Транзакция: `consents` → `leads` → `lead_events(created)` → `notification_outbox` (по одной записи на включённый канал).
8. Ответ `201 {"public_number": 123}`.

7.5. IP-адрес

- Реальный IP берётся из `X-Forwarded-For` **только** от доверенного прокси (Caddy) — через `--proxy-headers --forwarded-allow-ips`.
- В базе IP хранится только как `HMAC-SHA256(ip, SECRET_KEY)`. Открытый IP нигде не сохраняется и не логируется.

7.6. Rate limit на PostgreSQL

Фиксированное окно, атомарно, работает при нескольких воркерах:

```
async def hit(session: AsyncSession, bucket: str, limit: int, window_s: int) -> int | None:
    """Возвращает None, если лимит не превышен, иначе секунды до сброса окна."""
    row = (await session.execute(text("""
        INSERT INTO rate_limits (bucket, window_start, hits)
        VALUES (:bucket, to_timestamp(floor(extract(epoch from now()) / :w) * :w), 1)
        ON CONFLICT (bucket, window_start)
        DO UPDATE SET hits = rate_limits.hits + 1
        RETURNING hits,
          ceil(extract(epoch from window_start) + :w - extract(epoch from now()))::int AS retry_after
    """), {"bucket": bucket, "w": window_s})).one()
    return None if row.hits <= limit else max(row.retry_after, 1)
```

Rate limit выполняется в **отдельной** короткой транзакции до основной, чтобы откат основной не откатывал счётчик.

7.7. Аутентификация админки

Модель: серверные сессии в БД + cookie. JWT и `localStorage` **не используются**: сессию можно отозвать мгновенно, токен недоступен JavaScript.

Вход `POST /api/v1/admin/auth/login`:

1. Rate limit по `login:<ip_hash>:<email>`.
2. Найти пользователя; проверить пароль `argon2.PasswordHasher().verify`. Если пользователя нет — всё равно выполнить проверку против фиктивного хеша (одинаковое время ответа).
3. Ошибка → `401` с общим текстом «Неверный email или пароль» (без уточнения, что именно неверно).
4. Если `check_needs_rehash` → перехешировать.
5. Создать сессию: `token = secrets.token_urlsafe(32)`, `csrf = secrets.token_urlsafe(32)`; в БД — только SHA-256 обоих.
6. Cookie: - `__Host-session=<token>; Path=/; Secure; HttpOnly; SameSite=Strict; Max-Age=<ttl>` - `__Host-csrf=<csrf>; Path=/; Secure; SameSite=Strict; Max-Age=<ttl>` (без HttpOnly — фронтенд читает и шлёт в заголовке) - Префикс `__Host-` гарантирует: только HTTPS, без атрибута Domain, Path=/. - В `dev` (http://localhost) префикс и `Secure` отключаются флагом окружения.
7. Обновить `last_login_at`. Ответ `204`.

Проверка запроса: зависимость `current_admin` ищет сессию по SHA-256 токена, проверяет `revoked_at IS NULL AND expires_at > now()`, продлевает `last_seen_at` не чаще раза в 5 минут.

CSRF (double submit): для всех методов, кроме GET/HEAD/OPTIONS, заголовок `X-CSRF-Token` должен совпадать с cookie `__Host-csrf` и с хешем в сессии. Иначе `403 csrf_failed`. Плюс `SameSite=Strict` как второй слой.

Создание администратора: регистрации нет. Только CLI:

```
docker compose exec api uv run python -m app.cli create-admin --email owner@example.ru
# пароль вводится интерактивно, минимум 12 символов
```

7.8. Логирование

- structlog, JSON в stdout, поля: `ts`, `level`, `event`, `request_id`, `path`, `status`, `duration_ms`.
- **Запрещено логировать:** имя, телефон, email, текст сообщения, IP, cookie, заголовок Authorization, пароли.
- Ошибки 5xx логируются со стек-трейсом.

7.9. Health

- `GET /api/v1/healthz` → `200 {"status":"ok"}` — процесс жив.
- `GET /api/v1/readyz` → `SELECT 1` к БД; `200` или `503`. Используется в healthcheck Docker.
- Оба эндпоинта не логируются на уровне INFO (иначе healthcheck засоряет логи).

8. Контракт API

Базовый префикс `/api/v1`. Все тела — JSON, UTF-8.

8.1. Публичные

Метод	Путь	Тело	Ответ
POST	`/public/leads`	`LeadCreate`	`201 {public_number}` / `422` / `429`

8.2. Админка (cookie-сессия; для изменяющих — `X-CSRF-Token`)

Метод	Путь	Тело / параметры	Ответ
POST	`/admin/auth/login`	`{email, password}`	`204` + cookie / `401` / `429`
POST	`/admin/auth/logout`	—	`204`, сессия отозвана, cookie очищены
GET	`/admin/auth/me`	—	`200 {id, email}` / `401`
POST	`/admin/auth/password`	`{current_password, new_password}`	`204`; все **другие** сессии отзываются
POST	`/admin/auth/sessions/revoke-all`	—	`204`; отзываются все сессии, включая текущую
GET	`/admin/leads`	`status`, `q`, `date_from`, `date_to`, `page` (≥1), `page_size` (10–100, default 25), `sort` (`-created_at` по умолчанию)	`200 {items, page, page_size, total}`
GET	`/admin/leads/{id}`	—	`200 {lead, events}` / `404`
PATCH	`/admin/leads/{id}`	`{status, version}`	`200 lead` / `409 version_conflict` / `404`
POST	`/admin/leads/{id}/notes`	`{text}` (1–2000)	`201 event`
GET	`/admin/leads/export.csv`	те же фильтры, что у списка	`200 text/csv`

Пагинация: номер страницы + общее количество. Для объёма одного бизнеса (тысячи заявок) это проще в UI и достаточно быстро; если объём вырастет до сотен тысяч — переход на keyset-пагинацию по `(created_at, id)`.

Поиск `q`: по имени (`ILIKE`), по телефону (цифры из `q` сравниваются с цифрами `phone_e164`), по номеру заявки (если `q` — число).

Экспорт CSV:

- Кодировка UTF-8 **с BOM**, разделитель `;` — так файл корректно открывается в русском Excel двойным кликом.
- Защита от CSV-инъекций: если значение ячейки начинается с `=`, `+`, `-`, `@`, табуляции или возврата каретки — перед ним ставится `'`.
- Колонки: Номер, Дата (в часовом поясе бизнеса), Имя, Телефон, Email, Услуга, Статус, Сообщение.
- Отдаётся потоком (`StreamingResponse`), имя файла `leads-YYYY-MM-DD.csv`.

Пример ответа `GET /admin/leads/{id}`:

```
{
  "lead": {
    "id": "0192f5c4-...",
    "public_number": 123,
    "name": "Иван",
    "phone_e164": "+79990000000",
    "email": null,
    "message": "Нужна консультация",
    "service_slug": "diagnostics",
    "status": "new",
    "version": 1,
```

```
    "created_at": "2026-10-20T08:15:00Z"
  },
  "events": [
    {"id": "...", "type": "created", "actor": null, "payload": {}, "created_at": "2026-10-20T08:15:00Z"}
  ]
}
```

9. Уведомления

9.1. Каналы

Интерфейс канала:

```python
class Channel(Protocol):
    name: str
    async def send_lead_created(self, lead: LeadView) -> None: ...
```

- `email` — aiosmtplib, TLS; письмо в двух вариантах (text + html) из Jinja2-шаблона. Тема: «Новая заявка № 123». В письме: имя, телефон, услуга, сообщение, кнопка «Открыть в админке» (ссылка на `ADMIN_ORIGIN/leads/<id>`).
- `max` — включается, если заданы `MAX_BOT_TOKEN` и `MAX_CHAT_ID`. Создание бота в MAX доступно только подтверждённым аккаунтам юрлиц и ИП, поэтому канал выключен по умолчанию. Конкретные эндпоинты и формат запросов брать **из официальной документации MAX для разработчиков на момент реализации**; адаптер изолирован, чтобы изменения API MAX не затрагивали остальной код.
- `log` — пишет событие в лог. Используется в `dev` / `test`, когда нет SMTP.

9.2. Воркер

Отдельный процесс (`python -m app.worker`), тот же образ, что у API.

```python
BACKOFF_S = [60, 300, 900, 3600, 21600]  # 1 мин, 5 мин, 15 мин, 1 ч, 6 ч

async def process_batch(session: AsyncSession, channels: dict[str, Channel]) -> int:
    async with session.begin():
        rows = (await session.execute(text("""
            SELECT id, channel, payload, attempts FROM notification_outbox
            WHERE status = 'pending' AND next_attempt_at <= now()
            ORDER BY created_at
            LIMIT 20
            FOR UPDATE SKIP LOCKED
        """))).all()
        for row in rows:
            try:
                lead = await load_lead_view(session, row.payload["lead_id"])
                await channels[row.channel].send_lead_created(lead)
                await mark_sent(session, row.id)
            except Exception as exc:  # noqa: BLE001 — любая ошибка канала = повтор
                await schedule_retry(session, row.id, row.attempts, exc, BACKOFF_S)
    return len(rows)
```

- `FOR UPDATE SKIP LOCKED` — несколько воркеров не возьмут одну задачу.
- После 5 неудачных попыток статус `failed`, в лог уровня `error`.
- Цикл: если пачка пустая — пауза 2 с; корректная остановка по SIGTERM (дорабатывает текущую пачку).

- Раз в час: удаление старых окон `rate_limits`, удаление истёкших сессий, обезличивание заявок старше `LEAD_RETENTION_DAYS` (раздел 14).

10. Фронтенд: дизайн-система (`packages/ui`)

10.1. Принципы

1. **Приложения не пишут собственные стили для базовых вещей.** Текст, кнопки, поля, отступы, сетка — только компоненты `@app/ui`. В `apps/*` допускаются стили лишь для компоновки секций, и только через токены.
2. **Все значения — из токенов.** Никаких «магических» `#3a7` или `13px` в компонентах. Линтер `stylelint` с правилом, запрещающим сырые цвета вне файла токенов.
3. **Ни одной строки текста без i18n.** Компоненты принимают ключи сообщений (`MessageKey`), а не готовые строки.
4. **Каждый компонент:** TypeScript-пропсы, все состояния, доступность (роль, aria, фокус), история в Storybook на каждое состояние, unit-тест.
5. **Композиция вместо конфигурации:** сложные компоненты собираются из простых (`Field` = `Label` + контрол + `HelpText` / `ErrorText`).

10.2. Токены

Файл `packages/ui/src/tokens/tokens.css` — CSS custom properties. Два уровня:

Базовые (примитивы):

```css
:root {
  /* цвет бренда генерируется из site.config.ts при сборке */
  --brand-50: oklch(97% 0.02 var(--brand-hue));
  --brand-500: oklch(60% 0.15 var(--brand-hue));
  --brand-600: oklch(53% 0.15 var(--brand-hue));
  --brand-700: oklch(46% 0.14 var(--brand-hue));
  --gray-0: #ffffff;  --gray-50: #f7f7f8;  --gray-100: #eeeef0;
  --gray-300: #c9c9ce; --gray-500: #8a8a93; --gray-700: #3f3f46; --gray-900: #18181b;
  --red-600: #dc2626; --green-600: #16a34a; --amber-600: #d97706;

  --space-1: 4px;  --space-2: 8px;  --space-3: 12px; --space-4: 16px;
  --space-5: 20px; --space-6: 24px; --space-8: 32px; --space-10: 40px;
  --space-12: 48px; --space-16: 64px; --space-20: 80px;

  --radius-sm: 6px; --radius-md: 10px; --radius-lg: 16px; --radius-full: 999px;

  --font-sans: "Inter", system-ui, -apple-system, "Segoe UI", Roboto, sans-serif;
  --font-mono: ui-monospace, "JetBrains Mono", monospace;

  --text-xs: 0.75rem; --text-sm: 0.875rem; --text-md: 1rem; --text-lg: 1.125rem;
  --text-xl: 1.25rem; --text-2xl: clamp(1.5rem, 1.2rem + 1vw, 1.875rem);
  --text-3xl: clamp(1.875rem, 1.4rem + 2vw, 2.5rem);
  --text-display: clamp(2.25rem, 1.5rem + 3.5vw, 3.75rem);
  --leading-tight: 1.15; --leading-normal: 1.5;

  --shadow-sm: 0 1px 2px rgb(0 0 0 / 0.06);
  --shadow-md: 0 4px 12px rgb(0 0 0 / 0.08);
  --shadow-lg: 0 12px 32px rgb(0 0 0 / 0.12);

  --duration-fast: 120ms; --duration-normal: 200ms;
  --ease-standard: cubic-bezier(0.2, 0, 0, 1);

  --z-header: 100; --z-drawer: 400; --z-modal: 500; --z-toast: 600;
}
```

Семантические (ими пользуются компоненты):

```
:root, [data-theme="light"] {
  --color-bg: var(--gray-50);
  --color-surface: var(--gray-0);
  --color-surface-raised: var(--gray-0);
  --color-text: var(--gray-900);
  --color-text-muted: var(--gray-500);
  --color-text-inverse: var(--gray-0);
  --color-border: var(--gray-100);
  --color-border-strong: var(--gray-300);
  --color-accent: var(--brand-600);
  --color-accent-hover: var(--brand-700);
  --color-accent-subtle: var(--brand-50);
  --color-danger: var(--red-600);
  --color-success: var(--green-600);
  --color-warning: var(--amber-600);
  --color-focus-ring: var(--brand-500);
}
[data-theme="dark"] { /* те же семантические имена, тёмные значения */ }
@media (prefers-reduced-motion: reduce) {
  :root { --duration-fast: 0ms; --duration-normal: 0ms; }
}
```

Брейкпоинты (в медиазапросах, CSS-переменные в `@media` не работают): `sm 640px`, `md 768px`, `lg 1024px`, `xl 1280px`. Хранятся в `breakpoints.ts` и подставляются через PostCSS `@custom-media`.

Контраст (`MUST`): все пары «текст/фон» из семантических токенов проходят WCAG 2.2 AA (4.5:1 для обычного текста, 3:1 для крупного и для рамок полей). Проверяется тестом `tokens.contrast.test.ts`, который считает контраст для каждой пары и падает при нарушении. Цвет бренда из конфига подбирается так, чтобы `--color-accent` на белом давал ≥ 4.5:1; если нет — сборка падает с понятной ошибкой.

Шрифты: самостоятельно хостятся (без Google Fonts — нет запросов к иностранным серверам и вопросов по 152-ФЗ), `woff2`, подмножества `latin` + `cyrillic`, `font-display: swap`, `preload` для основного начертания.

10.3. Каталог компонентов

Ниже — полный список, пропсы и обязательные состояния. Имена пропсов фиксированы.

Типографика

`Text` — единственный способ вывести текст. Полная спецификация в разделе 11.

`Heading` — заголовок, где **семантический уровень отделён от внешнего вида**:

Проп	Тип	По умолчанию
`k`	`MessageKey`	— (обязателен)
`args`	`Record<string, unknown> | unknown[]`	—
`level`	`1 | 2 | 3 | 4`	— (обязателен) — рендерит `h1` – `h4`
`variant`	`'display' | 'h1' | 'h2' | 'h3'`	соответствует `level`
`align`	`'start' | 'center'`	`start`

На странице ровно один `level=1`. Внутри — `Text`.

Действия

`Button`

Проп	Тип	По умолчанию
`label`	`MessageKey`	обязателен, если нет `iconOnly`
`variant`	`'primary' | 'secondary' | 'ghost' | 'danger' | 'link'`	`primary`
`size`	`'sm' | 'md' | 'lg'`	`md`
`type`	`'button' | 'submit' | 'reset'`	`button`
`iconStart` / `iconEnd`	`IconName`	—
`iconOnly`	`IconName`	—
`ariaLabel`	`MessageKey`	обязателен при `iconOnly`
`loading`	`boolean`	`false`
`disabled`	`boolean`	`false`
`block`	`boolean`	`false` (на всю ширину)
`href` / `to`	`string` / `RouteLocationRaw`	— (рендерит `a` / `RouterLink`)

Обязательность `label` или пары `iconOnly + ariaLabel` задаётся **дискриминированным union-типом** — неправильная комбинация не компилируется.

Состояния и поведение:

- `default` / `:hover` / `:active` (сдвиг 1px вниз) / `:focus-visible` (кольцо 2px `--color-focus-ring`, отступ 2px; **никогда** `outline: none` без замены) / `disabled` (непрозрачность 0.5, `cursor: not-allowed`, клики не проходят) / `loading`.
- `loading`: текст остаётся в разметке, но прозрачный (ширина кнопки не прыгает), поверх `Spinner`, `aria-busy="true"`, повторные клики игнорируются, `disabled` на `button`.
- Минимальная зона нажатия 44×44 px на сенсорных экранах (для `sm` — за счёт псевдоэлемента).
- `type` по умолчанию `button` — чтобы кнопка внутри формы случайно её не отправляла.

`Link` — `k`, `href` | `to`, `external` (тогда `target="_blank" rel="noopener noreferrer"` и визуально скрытый текст «откроется в новой вкладке» из i18n), `tone`.

`IconButton` — обёртка `Button` с `iconOnly`, всегда квадратная.

Поля ввода

`Field` — контейнер поля: связывает подпись, контрол, подсказку и ошибку.

Проп	Тип
`label`	`MessageKey`
`hint`	`MessageKey`
`error`	`MessageKey` (код ошибки маппится на ключ `errors.field.<code>`)

Проп	Тип
`required`	`boolean`
`optionalMark`	`boolean` — показывает «(необязательно)»

`Field` генерирует `id` (через `useId()` из Vue 3.5) и передаёт в слот `{ id, describedBy, invalid }`. Контрол обязан выставить `id`, `aria-describedby`, `aria-invalid`. Ошибка выводится в элементе с `role="alert"` **только после** первого blur или попытки отправки.

`Input` — `v-model`, `type` (`text`, `email`, `tel`, `password`, `search`), `placeholder` (`MessageKey`), `autocomplete` (обязателен — для имени `name`, телефона `tel`, email `email`, пароля `current-password` / `new-password`), `inputmode`, `maxlength`, `readonly`, `disabled`, `invalid`, `prefixIcon`, `clearable`. Состояния: default, hover, focus, invalid (рамка `--color-danger`), disabled, readonly.

`PasswordInput` — `Input` + `IconButton` «показать/скрыть» (`aria-pressed`, ключи `common.showPassword` / `common.hidePassword`).

`PhoneInput` — маска `+7 (___) ___-__-__`:

- При вводе цифры подставляются в маску; `8` или `7` в начале заменяются на `+7`.
- Вставка из буфера любого формата (`89991234567`, `+7 999 123-45-67`, `9991234567`) нормализуется.
- Backspace удаляет цифру, а не символ маски.
- `inputmode="tel"`, `autocomplete="tel"`.
- `v-model` хранит **только цифры с кодом** (`79991234567`), отображение — с маской.
- Реализация — собственный композабл `usePhoneMask` в `packages/ui` с unit-тестами на 15+ случаев ввода и вставки.

`Textarea` — `v-model`, `rows`, `maxlength`, `autoresize` (до `maxRows`), счётчик символов «120 / 2000» (через `Text` с `args`), при приближении к лимиту (90%) счётчик меняет тон на `warning`.

`Select` — нативный `select` (лучшая доступность и поведение на мобильных), стилизованный. `options: {value: string; label: MessageKey}[]`, `placeholder` (`MessageKey`), `v-model`.

`Checkbox` — нативный `input type=checkbox` с кастомным видом; `v-model`, слот для подписи (чтобы внутри можно было поставить `Link`). **Никогда не отмечен по умолчанию, если это согласие.**

`SearchInput` — `Input type=search` + иконка + кнопка очистки + debounce (проп `debounceMs`, по умолчанию 300).

Компоновка

Компонент	Пропсы	Назначение
`Container`	`size: 'sm' | 'md' | 'lg' | 'xl'`	Максимальная ширина + боковые отступы `--space-4` (мобильный) / `--space-6`
`Stack`	`gap: SpaceToken`, `align`, `as`	Вертикальный flex с отступами
`Inline`	`gap`, `align`, `justify`, `wrap`	Горизонтальный flex
`Grid`	`cols: {base, md, lg}`, `gap`	Адаптивная сетка
`Surface`	`padding`, `radius`, `shadow`, `tone`, `as`	Карточка/подложка
`Section`	`id`, `tone: 'default' | 'muted' | 'accent'`, `spacing`	Секция лендинга с вертикальными отступами

Компонент	Пропсы	Назначение
`Divider`	`orientation`	Разделитель
`VisuallyHidden`	—	Текст только для скринридеров

`SpaceToken = 1 | 2 | 3 | 4 | 5 | 6 | 8 | 10 | 12 | 16 | 20` — только значения из шкалы.

Обратная связь и данные

Компонент	Ключевые пропсы	Поведение
`Spinner`	`size`, `label: MessageKey`	`role="status"`, текст для скринридеров
`Alert`	`tone: info/success/warning/danger`, `title`, `k`, `args`, `dismissible`	`role="alert"` для danger, иначе `role="status"`
`Toast` + `useToast()`	`toast.success(k, args)`, `toast.error(k, args)`, `action`	Стек справа снизу, автоскрытие 5 с (ошибки — 8 с), пауза при наведении и фокусе, `aria-live="polite"`
`Badge`	`tone`, `k`	Статусы заявок
`Skeleton`	`width`, `height`, `lines`	Заглушка при загрузке; анимация отключается при `prefers-reduced-motion`
`EmptyState`	`icon`, `title`, `description`, слот действия	Пустые списки
`ErrorState`	`code` (из problem+json), `onRetry`	Ошибка загрузки с кнопкой «Повторить»
`Dialog`	`open`, `title`, `description`, `confirmLabel`, `cancelLabel`, `tone`, `loading`	Нативный `<dialog>` + `showModal()`: фокус-ловушка и Esc из коробки; фокус возвращается на кнопку-источник
`Drawer`	`open`, `side`, `title`	Панель сбоку на `<dialog>`, на мобильном — на весь экран
`Accordion`	`items: {id, title: MessageKey, body: MessageKey}[]`	На нативных `<details>/<summary>`
`DataTable`	`columns`, `rows`, `loading`, `rowKey`, `onRowClick`, слоты ячеек	`<table>` с `<caption>` (визуально скрыт), строка кликабельна и доступна с клавиатуры (Enter), sticky-заголовок, на мобильном — карточки
`Pagination`	`page`, `pageSize`, `total`	Кнопки «Назад/Вперёд», номера, «Показано 1–25 из 340» через `Text` с `args`, `aria-current="page"`
`Icon`	`name: IconName`, `size`, `label?`	Встроенный SVG-спрайт; без `label` — `aria-hidden="true"`

Формы

`useForm<TSchema>()` — композабл в `packages/ui`, без внешних библиотек форм:

```ts
const form = useForm({
  schema: leadSchema,            // zod-схема
  initial: { name: '', phone: '', email: '', serviceSlug: '', message: '', consent: false },
  onSubmit: async (values) => api.createLead(values),
})
// form.values, form.errors, form.touched, form.isSubmitting, form.submitCount
// form.field('name') -> { modelValue, 'onUpdate:modelValue', onBlur, error, invalid }
// form.setServerErrors(problem.errors)  // маппинг кодов сервера на поля
// form.handleSubmit                     // валидирует, фокусирует первое невалидное поле
```

- Ошибка поля показывается после blur этого поля или после первой попытки отправки.

- При отправке с ошибками фокус ставится на первое невалидное поле, над формой появляется `Alert` со списком ошибок-ссылок (каждая ведёт к полю).
- Двойная отправка невозможна: пока `isSubmitting`, `handleSubmit` ничего не делает.

11. i18n и компонент `Text`

11.1. Словарь

`packages/i18n/src/ru.json` — единственный источник текстов. Ключи иерархические, по месту использования:

```
{
  "common": {
    "required": "обязательно",
    "optional": "необязательно",
    "showPassword": "Показать пароль",
    "hidePassword": "Скрыть пароль",
    "opensInNewTab": "откроется в новой вкладке",
    "retry": "Повторить",
    "cancel": "Отмена"
  },
  "lead": {
    "form": {
      "title": "Оставьте заявку",
      "subtitle": "Перезвоним в течение {minutes} минут в рабочее время",
      "name": "Как к вам обращаться",
      "phone": "Телефон",
      "submit": "Отправить заявку",
      "messageCounter": "{count} / {max}",
      "success": "Заявка № {number} принята. Скоро свяжемся!"
    }
  },
  "admin": {
    "leads": {
      "shown": "Показано {from}–{to} из {total}",
      "count": "нет заявок | {n} заявка | {n} заявки | {n} заявок"
    }
  },
  "errors": {
    "field": {
      "required": "Заполните это поле",
      "too_short": "Слишком коротко",
      "phone_invalid": "Проверьте номер телефона",
      "consent_required": "Нужно согласие на обработку данных"
    },
    "api": {
      "rate_limited": "Слишком много попыток. Попробуйте через {minutes} мин.",
      "internal_error": "Что-то пошло не так. Попробуйте ещё раз",
      "network": "Нет соединения с сервером"
    }
  }
}
```

Плюрализация для русского языка (одна/две/пять форм) настраивается через `pluralRules` в vue-i18n: функция выбора формы для `ru`.

11.2. Типизированные ключи

```
// packages/i18n/src/index.ts
import { createI18n } from 'vue-i18n'
import ru from './ru.json'
```

```ts
type Leaves<T, P extends string = ''> = {
  [K in keyof T & string]: T[K] extends Record<string, unknown>
    ? Leaves<T[K], `${P}${K}.`>
    : `${P}${K}`
}[keyof T & string]

export type MessageKey = Leaves<typeof ru>

export const i18n = createI18n({
  legacy: false,
  locale: 'ru',
  fallbackLocale: 'ru',
  messages: { ru },
  pluralRules: { ru: ruPluralRule },
  missingWarn: import.meta.env.DEV,
  fallbackWarn: import.meta.env.DEV,
})

/** Для скриптов: T('lead.form.success', { number: 12 }) или T('key', [a, b]) */
export function T(key: MessageKey, args?: Record<string, unknown> | unknown[]): string {
  return args === undefined
    ? i18n.global.t(key)
    : Array.isArray(args)
      ? i18n.global.t(key, args)
      : i18n.global.t(key, args)
}
```

Опечатка в ключе — **ошибка компиляции**.

11.3. Компонент `Text`

```vue
<!-- packages/ui/src/components/Text/Text.vue -->
<script setup lang="ts">
import { computed } from 'vue'
import { useI18n } from 'vue-i18n'
import type { MessageKey } from '@app/i18n'

export type TextVariant =
  | 'display' | 'h1' | 'h2' | 'h3'
  | 'body-lg' | 'body' | 'body-sm' | 'caption' | 'label' | 'code'
export type TextTone =
  | 'default' | 'muted' | 'accent' | 'danger' | 'success' | 'warning' | 'inverse'

const props = withDefaults(defineProps<{
  k: MessageKey
  args?: Record<string, unknown> | unknown[]
  plural?: number
  as?: 'span' | 'p' | 'div' | 'strong' | 'em' | 'label' | 'small' | 'li' | 'dt' | 'dd'
  variant?: TextVariant
  tone?: TextTone
  weight?: 'regular' | 'medium' | 'semibold' | 'bold'
  align?: 'start' | 'center' | 'end'
  lines?: number
}>(), {
  as: 'span',
  variant: 'body',
  tone: 'default',
  align: 'start',
})

const { t } = useI18n()

const content = computed(() => {
  if (props.plural !== undefined) {
    // сигнатура vue-i18n: t(key, named, plural)
    const named = Array.isArray(props.args) ? {} : (props.args ?? {})
    return t(props.k, { n: props.plural, ...named }, props.plural)
  }
```

```
  if (props.args === undefined) return t(props.k)
  return Array.isArray(props.args) ? t(props.k, props.args) : t(props.k, props.args)
})

const classes = computed(() => [
  'ui-text',
  `ui-text--${props.variant}`,
  `ui-text--tone-${props.tone}`,
  props.weight && `ui-text--w-${props.weight}`,
  `ui-text--align-${props.align}`,
  props.lines && 'ui-text--clamp',
])
</script>

<template>
  <component
    :is="as"
    :class="classes"
    :style="lines ? { '--ui-text-lines': lines } : undefined"
  >{{ content }}</component>
</template>

<style scoped>
.ui-text { margin: 0; font-family: var(--font-sans); color: var(--color-text); }
.ui-text--body    { font-size: var(--text-md); line-height: var(--leading-normal); }
.ui-text--body-sm { font-size: var(--text-sm); line-height: var(--leading-normal); }
.ui-text--caption { font-size: var(--text-xs); line-height: var(--leading-normal); }
.ui-text--h1      { font-size: var(--text-3xl); line-height: var(--leading-tight); font-weight: 700; }
.ui-text--display { font-size: var(--text-display); line-height: var(--leading-tight); font-weight:
800; }
.ui-text--tone-muted  { color: var(--color-text-muted); }
.ui-text--tone-danger { color: var(--color-danger); }
.ui-text--clamp {
  display: -webkit-box;
  -webkit-box-orient: vertical;
  -webkit-line-clamp: var(--ui-text-lines);
  overflow: hidden;
}
/* остальные варианты, тона и веса — по тому же принципу, только через токены */
</style>
```

Использование:

```
<Text k="lead.form.subtitle" :args="{ minutes: 15 }" as="p" tone="muted" />
<Text k="admin.leads.count" :plural="total" variant="body-sm" />
<Text k="admin.leads.shown" :args="{ from: 1, to: 25, total: 340 }" />
```

Правило текста в HTML: vue-i18n не интерпретирует HTML в сообщениях, `v-html` в проекте **запрещён** (правило ESLint `vue/no-v-html: error`). Если в предложение нужно вставить ссылку (например, в согласие), используется компонент `i18n-t` с именованными слотами:

```
<i18n-t keypath="lead.form.consent" tag="span">
  <template #policy><Link k="lead.form.policyLink" to="/privacy" /></template>
  <template #consent><Link k="lead.form.consentLink" to="/consent" /></template>
</i18n-t>
```

11.4. Проверки (MUST)

- ESLint: `vue/no-bare-strings-in-template: error` — сырые строки в шаблонах запрещены.
- Скрипт `pnpm i18n:check`: находит неиспользуемые ключи и ключи, которых нет в словаре (падает в CI).

12. Публичный сайт (`apps/site`)

12.1. Конфиг контента

`apps/site/site.config.ts` — единый источник бренда и контента. Тексты контента кладутся в отдельный словарь `content.ru.json` того же пакета i18n (ключи `content.*`), чтобы правило «никаких строк в шаблонах» соблюдалось и для контента.

```
export default defineSiteConfig({
  demo: true,
  brand: { name: 'content.brand.name', hue: 250, logo: '/logo.svg' },
  contacts: {
    phoneE164: '+70000000000',
    email: 'hello@example.ru',
    address: 'content.contacts.address',
    workingHours: 'content.contacts.hours',
    timezone: 'Asia/Novosibirsk',
  },
  services: [
    { slug: 'service-a', title: 'content.services.a.title', description: 'content.services.a.desc', pric
eFrom: 1500, icon: 'wrench' },
  ],
  advantages: [ /* ... */ ],
  process: [ /* шаги: заявка → звонок → работа → результат */ ],
  faq: [ /* только реальные вопросы бизнеса */ ],
  seo: { title: 'content.seo.title', description: 'content.seo.description', ogImage: '/og.png' },
  analytics: { yandexMetrikaId: null },   // null = аналитики нет, cookie-баннер не нужен
  leadForm: { callbackMinutes: 15 },
})
```

- `defineSiteConfig` валидирует конфиг zod-схемой **при сборке**; ошибка конфига = упавшая сборка с понятным сообщением.
- При сборке из `services` генерируется `services.json` (slug-и), который копируется в образ API для проверки `service_slug` (раздел 7.4).

12.2. Структура главной

Порядок секций и что в каждой — **до последней кнопки**:

1. **`SiteHeader`** (липкий, фон `--color-surface`, тень после прокрутки на 8px)

- Логотип (ссылка на `/`, `aria-label` = название бренда).
- Навигация-якоря: Услуги, Цены, Как мы работаем, Вопросы, Контакты. Активный пункт подсвечивается при прокрутке (IntersectionObserver), `aria-current="true"`.
- Телефон — `Link` с `href="tel:+7..."`, отображение с маской.
- Кнопка `Button variant="primary" size="sm" label="nav.cta"` («Оставить заявку») — плавная прокрутка к форме и фокус на первое поле.
- На мобильном (< md): логотип + `IconButton` «Меню» (`aria-expanded`, `aria-controls`) → `Drawer` с навигацией, телефоном и CTA. Закрытие: крестик, Esc, клик по пункту.

2. **`HeroSection`**

- `Heading level=1 variant="display"` — главный оффер.
- `Text variant="body-lg" tone="muted"` — подзаголовок.
- Кнопки: `primary lg` «Оставить заявку» (к форме) и `secondary lg` «Позвонить» (`tel:`; на десктопе показывает номер).

- Изображение: `<picture>` AVIF + WebP + JPEG, явные `width/height` (нет сдвига вёрстки), `fetchpriority="high"` (это LCP-элемент), без `loading="lazy"`.

3. **`ServicesSection`** — `Grid cols={base:1, md:2, lg:3}` карточек `Surface`: иконка, `Heading level=3`, описание, «от 1 500 ₽» (через `Intl.NumberFormat('ru-RU')` и `Text` с `args`), кнопка `ghost` «Выбрать» → прокрутка к форме **с предвыбранной услугой** в `Select`.

4. **`AdvantagesSection`** — 3–4 пункта: иконка + заголовок + текст.

5. **`PricesSection`** — таблица `DataTable` (без клика по строкам): услуга / цена «от». Под таблицей `Text tone="muted"` — оговорка, что итоговая цена после диагностики/замера.

6. **`ProcessSection`** — нумерованные шаги (`ol`), горизонтально на десктопе, вертикально на мобильном.

7. **`FaqSection`** — `Accordion`; первый пункт раскрыт. JSON-LD `FAQPage` генерируется из тех же данных.

8. **`ContactsSection`** — адрес, часы работы, телефон, email. Карта: **по умолчанию статичная картинка с кнопкой «Показать карту»**; интерактивная Яндекс.Карта грузится только после клика (сторонний скрипт и cookie — только по действию пользователя).

9. **`LeadFormSection`** — раздел 12.3.

10. **`SiteFooter`** — бренд, контакты, ссылки «Политика обработки персональных данных» и «Согласие на обработку персональных данных», копирайт с текущим годом, плашка `Badge` «Демо-проект» при `demo: true`.

11. **`CookieBanner`** — **только** если в конфиге включена аналитика. Кнопки «Принять» и «Отклонить» равнозначные по виду. До согласия скрипт аналитики не загружается. Выбор хранится в `localStorage` (обёрнут в try/catch).

12.3. Форма заявки — полная спецификация

Поля:

Поле	Компонент	Обязательно	Правила (клиент = сервер)	autocomplete
Имя	`Input`	да	2–100 символов после trim	`name`
Телефон	`PhoneInput`	да	11 цифр, начинается с 7	`tel`
Email	`Input type=email`	нет	валидный email	`email`
Услуга	`Select`	нет	slug из конфига; опция по умолчанию «Не знаю, нужна консультация»	—
Сообщение	`Textarea`	нет	≤ 2000, счётчик	—
Согласие	`Checkbox`	да	должно быть отмечено; **не отмечено по умолчанию**	—
`website`	скрытое `Input`	—	honeypot: вне экрана, `tabindex="-1"`, `aria-hidden="true"`, `autocomplete="off"`	`off`

Текст согласия: «Я даю согласие на обработку персональных данных в соответствии с [политикой] и [условиями согласия]» — через `i18n-t` со ссылками (раздел 11.3).

`elapsed_ms` считается от монтирования формы до отправки. UTM-метки читаются из URL при загрузке страницы и держатся в памяти (не в `localStorage`).

Состояния формы:

Состояние	Что видит пользователь
`idle`	Пустая форма, кнопка «Отправить заявку» активна
`invalid`	Ошибки под полями (после blur или попытки отправки), `Alert` над формой со списком ссылок на поля, фокус на первом ошибочном поле
`submitting`	Кнопка в `loading`, поля `readonly`, повторная отправка невозможна
`success`	Форма заменяется панелью: иконка-галочка, `Heading` «Заявка № 123 принята», текст «Перезвоним в течение 15 минут в рабочее время», кнопка `secondary` «Отправить ещё одну» (сбрасывает форму). Фокус переносится на заголовок панели (`tabindex="-1"`), чтобы скринридер озвучил результат. Цель аналитики `lead_submitted` — только если аналитика включена и разрешена
`server-validation`	Ошибки с сервера (`422`) раскладываются по полям через `setServerErrors`
`rate-limited`	`Alert tone="warning"`: «Слишком много попыток. Попробуйте через N мин.» (N из `Retry-After`), кнопка неактивна до истечения времени
`network-error`	`Alert tone="danger"`: «Нет соединения с сервером», данные в форме сохранены, кнопка «Повторить»
`server-error`	`Alert tone="danger"` с текстом по коду `internal_error` и ссылкой «позвонить напрямую» (`tel:`)

Таймаут запроса: 15 секунд (`AbortController`), после — состояние `network-error`.

12.4. Страницы

- `/privacy` — политика обработки ПДн (шаблон-текст из `content.legal.privacy`, версия = `PRIVACY_POLICY_VERSION`, дата вступления).
- `/consent` — текст согласия на обработку ПДн.
- `404` — `Heading`, текст, кнопка «На главную». Caddy отдаёт её со статусом 404.

12.5. SEO

- Пререндер всех страниц в статический HTML (vite-ssg).
- `<html lang="ru">`, `<title>` и `meta description` из конфига, `link rel="canonical"`, Open Graph и Twitter-карточка, `og:image` 1200×630.
- JSON-LD: `LocalBusiness` (только реальные данные из конфига; в демо — тестовые данные с пометкой) и `FAQPage`.
- `robots.txt` и `sitemap.xml` генерируются при сборке. При `demo: true` в `robots.txt` — `Disallow: /`, чтобы демо не индексировалось как настоящий бизнес.
- Favicon: SVG + PNG 180×180 (apple-touch-icon) + `site.webmanifest`.

12.6. Производительность (MUST, проверяется Lighthouse CI на мобильном профиле)

- Performance ≥ 95, Accessibility = 100, Best Practices ≥ 95, SEO = 100.
- LCP < 2.5 с, CLS < 0.1, INP < 200 мс.

- JS первой загрузки сайта ≤ 100 КБ gzip. Форма и `Drawer` подгружаются лениво (`defineAsyncComponent`).
- Картинки: AVIF/WebP, `srcset` / `sizes`, `loading="lazy"` для всего ниже первого экрана.
- Caddy: `encode zstd gzip`; для ассетов с хешем в имени `Cache-Control: public, max-age=31536000, immutable`; для HTML — `no-cache`.

13. Админка (`apps/admin`)

13.1. Общее

- SPA: Vue Router (history mode), Pinia (только UI-состояние и текущий пользователь), `@tanstack/vue-query` (всё, что приходит с сервера).
- Тема: светлая/тёмная по `prefers-color-scheme` + переключатель в шапке (выбор в `localStorage`, try/catch).
- Обёртка API (`packages/api-client`): генерируемые типы + `openapi-fetch`; middleware добавляет `X-CSRF-Token` из cookie `__Host-csrf` ко всем изменяющим запросам и превращает ответы problem+json в типизированную ошибку `ApiError {status, code, errors, retryAfter}`.
- **Глобальная обработка `401`:** очистка кэша запросов → редирект на `/login?next=<текущий путь>` → toast «Сессия истекла, войдите снова». Параметр `next` принимается только как относительный путь внутри админки (защита от открытого редиректа).
- Маршруты защищены guard'ом: перед входом на защищённый маршрут — `GET /auth/me` (кэшируется).
- Все фильтры списка синхронизированы с URL query — ссылку на отфильтрованный список можно сохранить и отправить.
- Мета: `<meta name="robots" content="noindex, nofollow">`.

13.2. Экран «Вход» (`/login`)

- По центру `Surface`: логотип, `Heading level=1` «Вход в админку».
- `Field` + `Input type=email autocomplete=username`, `Field` + `PasswordInput autocomplete=current-password`.
- Кнопка `primary block` «Войти» (`type=submit`, Enter отправляет).
- Ошибки: `401` → `Alert danger` «Неверный email или пароль» (поля не подсвечиваются по отдельности); `429` → «Слишком много попыток. Попробуйте через N мин.».
- После успеха — переход на `next` или `/leads`.
- Ссылки «Забыли пароль?» нет: сброс пароля в MVP — через CLI (описано в README).

13.3. Каркас `AppShell`

- Слева `nav` (на мобильном — `Drawer` по кнопке «Меню»): «Заявки» (с `Badge` — число новых, обновляется каждые 60 с, пока вкладка активна), «Аккаунт».
- Сверху: название бизнеса, переключатель темы (`IconButton`, `aria-pressed`), email пользователя, кнопка `ghost` «Выйти».
- «Выйти» → `POST /auth/logout` → очистка кэша → `/login`.
- Ссылка «Перейти к содержимому» (skip link) первой в порядке табуляции.

13.4. Экран «Заявки» (`/leads`)

Панель инструментов:

- `SearchInput` «Поиск по имени, телефону или номеру» (debounce 300 мс, сбрасывает страницу на 1).
- Фильтр статуса — `Select`: Все, Новые, В работе, Выполнены, Спам, Архив.
- Период — два `Input type=date` «с» и «по» (`date_to` не раньше `date_from`, иначе ошибка поля).
- Кнопка `ghost` «Сбросить фильтры» — видна, только когда есть активные фильтры.
- Кнопка `secondary` с иконкой «Скачать CSV» — выгрузка с текущими фильтрами; во время загрузки `loading`.

Таблица `DataTable`:

Колонка	Содержимое
№	`public_number`
Дата	`20.10.2026, 15:15` в часовом поясе бизнеса (`Intl.DateTimeFormat`)
Имя	обрезается до 1 строки с `title`
Телефон	с маской + `IconButton` «Скопировать» (toast «Телефон скопирован»)
Услуга	название из словаря по slug или «—»
Статус	`Badge`: new — accent, in_progress — warning, done — success, spam — danger, archived — muted

- Клик или Enter по строке → открывает `Drawer` с карточкой заявки (URL меняется на `/leads/<id>` — карточку можно открыть по прямой ссылке).
- Новые заявки (`status=new`) выделены полужирным.
- Сортировка: по дате (по умолчанию — новые сверху), переключается по клику на заголовок «Дата» (`aria-sort`).

Состояния списка:

- Загрузка — `Skeleton` строк (10 шт.).
- Пусто без фильтров — `EmptyState` «Заявок пока нет. Как только кто-то оставит заявку на сайте, она появится здесь».
- Пусто с фильтрами — `EmptyState` «Ничего не найдено» + кнопка «Сбросить фильтры».
- Ошибка — `ErrorState` с кнопкой «Повторить».
- Под таблицей — `Pagination` и «Показано 1–25 из 340».

13.5. Карточка заявки (`/leads/:id`, в `Drawer`)

- Заголовок: «Заявка № 123», дата создания, `Badge` статуса.
- **Контакт:** имя; телефон + кнопки «Позвонить» (`tel:`), «Скопировать»; email (если есть) + «Написать» (`mailto:`); услуга; сообщение (с сохранением переносов строк, как обычный текст).
- **Статус:** `Select` со статусами. Изменение применяется сразу (`PATCH` с текущим `version`):
- перевод в «Спам» требует подтверждения в `Dialog` («Отметить как спам?» / «Отметить» / «Отмена»);
- во время запроса `Select` заблокирован;
- успех → toast «Статус изменён», обновление списка и счётчика новых;

- `409 version_conflict` → toast «Заявку уже изменили. Показана актуальная версия», карточка перезагружается.
- **Заметки:** `Textarea` «Добавьте заметку» + кнопка `secondary` «Добавить заметку» (неактивна при пустом тексте; Ctrl/Cmd+Enter отправляет). После добавления поле очищается, заметка появляется в истории.
- **История:** лента `lead_events` в обратном порядке: «Заявка создана», «Статус: Новая → В работе (owner@example.ru)», заметки с автором и временем.
- Закрытие `Drawer`: крестик, Esc, клик по подложке → возврат на `/leads` с сохранёнными фильтрами.

13.6. Экран «Аккаунт» (`/account`)

- **Смена пароля:** «Текущий пароль», «Новый пароль» (минимум 12 символов, индикатор длины), «Повторите новый пароль». Кнопка «Сменить пароль». Успех → toast «Пароль изменён. Остальные сеансы завершены».
- **Безопасность:** кнопка `danger` «Выйти на всех устройствах» → `Dialog` с подтверждением → `POST /auth/sessions/revoke-all` → `/login`.

14. Персональные данные (152-ФЗ)

Проект собирает имя, телефон и email — это персональные данные. С 30 мая 2025 года штрафы за нарушения выросли кратно, Роскомнадзор проверяет сайты автоматическим сканером. Поэтому эти пункты — **MUST**, а не пожелания:

1. **Согласие:** отдельный чекбокс, **не отмеченный по умолчанию** (сканер РКН проверяет атрибут `checked`), со ссылками на политику и текст согласия. Без согласия форма не отправляется (проверка и на клиенте, и на сервере).
2. **Фиксация согласия:** таблица `consents` — версия политики, хеш показанного текста, время, хеш IP, user agent.
3. **Политика обработки ПДн** опубликована на `/privacy` и доступна из подвала и из формы.
4. **Локализация:** база данных на сервере в России (VDS Beget — российский хостинг).
5. **Без сторонних трекеров без согласия:** никаких внешних шрифтов, виджетов и счётчиков до согласия пользователя. Яндекс.Метрика и карта — только после явного действия.
6. **Минимизация:** IP хранится только как HMAC; персональные данные не пишутся в логи и не кладутся в очередь уведомлений.
7. **Срок хранения:** заявки старше `LEAD_RETENTION_DAYS` обезличиваются воркером: имя → «Удалено», телефон → `+70000000000`, email и сообщение → NULL, `anonymized_at` = now(). Статистика (даты, статусы, услуги) сохраняется.
8. **Уведомление в Роскомнадзор** о начале обработки ПДн — обязанность **оператора** (владельца бизнеса), подаётся через Госуслуги или сайт РКН **до** начала сбора данных. Для реального клиента это пункт чек-листа запуска в README; для демо с тестовыми данными — пометка в README.

Тексты политики и согласия в проекте — **шаблоны**, которые владелец бизнеса обязан проверить и адаптировать (лучше с юристом). Это явно указано в README.

15. Безопасность — чек-лист

Угроза	Мера
Кража сессии через XSS	Cookie `HttpOnly`; `v-html` запрещён; строгий CSP
CSRF	`SameSite=Strict` + double-submit токен в `X-CSRF-Token`
Перебор паролей	Rate limit на IP+email, Argon2id, одинаковое время ответа
Спам через форму	Rate limit, honeypot, проверка времени заполнения, опционально Yandex SmartCaptcha
SQL-инъекции	Только параметризованные запросы SQLAlchemy
CSV-инъекции	Экранирование `= + - @` в экспорте
Открытый редирект	`next` только относительный путь админки
Утечка секретов	`.env` в `.gitignore`, секреты только в env на сервере, `gitleaks` в pre-commit и CI
Уязвимые зависимости	`pip-audit` и `pnpm audit --prod` в CI; Dependabot/Renovate
Контейнеры	Пользователь без root, `read_only: true` где возможно, только нужные порты наружу (80/443)
Сервер	Вход по SSH-ключу, вход по паролю и root отключены, `ufw` (22, 80, 443), `fail2ban`, автообновления безопасности

Заголовки (Caddy, `MUST`):

```
Strict-Transport-Security: max-age=31536000; includeSubDomains
X-Content-Type-Options: nosniff
Referrer-Policy: strict-origin-when-cross-origin
Permissions-Policy: camera=(), microphone=(), geolocation=()
Content-Security-Policy: default-src 'self'; img-src 'self' data:; style-src 'self';
  script-src 'self'; connect-src 'self'; frame-ancestors 'none'; base-uri 'self';
  form-action 'self'
```

Если включены Метрика или карта, их домены добавляются в CSP точечно и только для сайта (не для админки). В CSP не используется `unsafe-inline` для скриптов; если vite-ssg требует инлайн-скрипт — через хеш (`sha256-...`).

16. Доступность

Цель — **WCAG 2.2 AA**.

- Вся навигация с клавиатуры; видимый фокус везде.
- Порядок заголовков без пропусков; один `h1` на страницу.
- Все поля имеют `label`; ошибки связаны через `aria-describedby`; `aria-invalid`.
- Модальные окна — нативный `<dialog>`: фокус-ловушка, Esc, возврат фокуса.
- Анимации отключаются при `prefers-reduced-motion`.
- Цвет не единственный носитель смысла: у статусов есть текст, у ошибок — иконка и текст.
- Проверка: axe в Storybook (каждая история), axe в Playwright (каждая страница), Lighthouse Accessibility = 100.

17. Тестовая среда

17.1. Пирамида

Уровень	Что	Инструменты	Где запускается
Unit (бэкенд)	Сервисы, валидация, нормализация телефона, CSV-экранирование, backoff	pytest	локально, CI
Интеграционные (бэкенд)	Роуты + настоящая БД: заявки, rate limit, авторизация, CSRF, outbox	pytest + httpx + testcontainers (PostgreSQL 18)	локально, CI
Контрактные	API соответствует своей OpenAPI-схеме, фаззинг входных данных	schemathesis	CI
Unit (фронтенд)	Компоненты UI, `usePhoneMask`, `useForm`, i18n	Vitest + @testing-library/vue + happy-dom	локально, CI
Визуальные / доступность компонентов	Каждое состояние каждого компонента	Storybook + addon-a11y + test-runner	CI
E2E	Полные сценарии через браузер	Playwright (Chromium + WebKit, мобильный вьюпорт)	CI на `compose.test.yml`
Производительность	Метрики сайта	Lighthouse CI	CI

17.2. Бэкенд: фикстуры

```python
# tests/conftest.py
import pytest
from testcontainers.postgres import PostgresContainer

@pytest.fixture(scope="session")
def pg_url():
    with PostgresContainer("postgres:18") as pg:
        yield pg.get_connection_url().replace("psycopg2", "asyncpg")

@pytest.fixture(scope="session")
async def engine(pg_url):
    engine = create_async_engine(pg_url)
    await run_migrations(pg_url)        # прогоняем настоящие миграции Alembic
    yield engine
    await engine.dispose()

@pytest.fixture
async def session(engine):
    # каждый тест — во вложенной транзакции, которая откатывается
    async with engine.connect() as conn:
        trans = await conn.begin()
        async with AsyncSession(bind=conn, join_transaction_mode="create_savepoint") as s:
            yield s
        await trans.rollback()

@pytest.fixture
async def client(session):
    app = create_app()
    app.dependency_overrides[get_session] = lambda: session
```

```
    async with AsyncClient(transport=ASGITransport(app=app), base_url="http://test") as c:
        yield c
```

- `pytest-asyncio` в режиме `asyncio_mode = "auto"`.
- Фабрики тестовых данных — `polyfactory`.
- Время в тестах замораживается (`time-machine`) там, где важны окна rate limit и backoff.
- Миграции тестируются отдельно: `upgrade head` → `downgrade base` → `upgrade head` на чистой БД.

Обязательные интеграционные тесты (минимум):

- Создание заявки: `201`, номер, запись в `consents`, `leads`, `lead_events`, `notification_outbox`.
- Без согласия → `422 consent_required`. Старая версия политики → `422`.
- Невалидный телефон → `422 phone_invalid`. Неизвестная услуга → `422 unknown_service`.
- Honeypot заполнен → `201`, но в БД **ничего нет**. `elapsed_ms < 3000` → то же самое.
- 6-я заявка с одного IP за 10 минут → `429` с `Retry-After`.
- Вход: неверный пароль → `401` с общим кодом; несуществующий email → тот же ответ; 11-я попытка → `429`.
- Изменяющий запрос без `X-CSRF-Token` → `403`; с неверным → `403`.
- Истёкшая и отозванная сессии → `401`.
- `PATCH` со старым `version` → `409`.
- Смена пароля отзывает другие сессии, текущая остаётся.
- Экспорт CSV: BOM, `;`, экранирование ячейки, начинающейся с `=`.
- Воркер: успешная отправка → `sent`; ошибка канала → `attempts+1` и `next_attempt_at` по таблице backoff; после 5 ошибок → `failed`; две параллельные обработки не берут одну задачу.
- Обезличивание: заявка старше срока обезличена, свежая — нет.
- В логах нет телефона и имени (тест перехватывает вывод structlog и ищет значения).

Пороги покрытия: `services/` и `security/` ≥ 95%, весь бэкенд ≥ 90% (`pytest --cov --cov-fail-under=90`).

17.3. Фронтенд

- Каждый компонент `packages/ui` — тест на рендер, пропсы, события, доступность (`vitest-axe`), клавиатуру.
- `usePhoneMask` — табличный тест: ввод по одной цифре, вставка 6 разных форматов, backspace на разделителе, ввод с `8`.
- `useForm` — показ ошибок после blur и после submit, фокус первого невалидного, блок двойной отправки, маппинг серверных ошибок.
- Storybook: история на **каждое состояние** (default, hover через параметр псевдосостояний, focus, disabled, loading, invalid, с длинным текстом, в тёмной теме). `test-runner` прогоняет все истории с axe.
- Пороги покрытия `packages/ui` ≥ 90%.

17.4. E2E (Playwright)

Окружение: `compose.test.yml` (db, api, worker, site, admin, caddy, mailpit). Перед прогоном — миграции и создание тестового администратора.

Сценарии (минимум):

1. Посетитель отправляет заявку с главной → видит «Заявка № N принята» → в Mailpit (через его HTTP API) пришло письмо с номером N.
2. Выбор услуги в карточке → форма прокручена, услуга предвыбрана.
3. Отправка пустой формы → ошибки, фокус на «Имя», `Alert` со ссылками.
4. Администратор входит → видит заявку из сценария 1 первой в списке → открывает → меняет статус на «В работе» → статус в истории.
5. Добавление заметки через Ctrl+Enter.
6. Фильтр «Новые» + поиск по телефону → URL содержит параметры → перезагрузка страницы сохраняет фильтры.
7. Скачивание CSV → файл начинается с BOM, содержит заявку.
8. «Выйти на всех устройствах» → редирект на вход; старая вкладка при следующем запросе уходит на вход.
9. Мобильный вьюпорт (390×844): меню открывается и закрывается, форма отправляется.
10. Каждая страница сайта и админки проходит axe без нарушений.

17.5. Команды

```
dev:        docker compose -f infra/compose.dev.yml up --build
test:       test-api test-web
test-api:   cd apps/api && uv run pytest --cov --cov-fail-under=90
test-web:   pnpm -r test -- --coverage
storybook:  pnpm --filter @app/ui storybook
e2e:        docker compose -f infra/compose.test.yml up -d --build --wait && pnpm --filter e2e test
lint:       cd apps/api && uv run ruff check . && uv run ruff format --check . ; pnpm -r lint
typecheck:  cd apps/api && uv run mypy --strict app ; pnpm -r typecheck
openapi:    cd apps/api && uv run python -m app.export_openapi > ../../packages/api-client/openapi.json
&& pnpm --filter @app/api-client generate
migrate:    docker compose exec api uv run alembic upgrade head
```

18. Качество кода

- **Python:** ruff (линтер + форматтер, включены наборы правил `E,F,I,B,UP,S,ASYNC,SIM,RUF`), mypy `--strict`, все функции с аннотациями типов.
- **TypeScript/Vue:** ESLint flat config (`typescript-eslint` strict, `eslint-plugin-vue` recommended + `vue/no-bare-strings-in-template`, `vue/no-v-html`, `vue/require-explicit-emits`, `vue/component-api-style: ['script-setup']`), Prettier, `vue-tsc --noEmit`, stylelint (запрет сырых цветов вне токенов).
- **TS-настройки:** `strict: true`, `noUncheckedIndexedAccess: true`, `exactOptionalPropertyTypes: true`.
- **Хуки:** lefthook — pre-commit: ruff, prettier, eslint на изменённых файлах, gitleaks; commit-msg: commitlint (Conventional Commits).
- **Импорты:** приложения импортируют UI только из `@app/ui` (публичный `index.ts`), не из внутренних путей пакета (правило `no-restricted-imports`).

19. Docker и инфраструктура

19.1. Образы

API — многоэтапная сборка:

```
FROM python:3.14-slim AS build
COPY --from=ghcr.io/astral-sh/uv:latest /uv /usr/local/bin/uv
WORKDIR /app
COPY pyproject.toml uv.lock ./
RUN uv sync --frozen --no-dev --no-install-project
COPY . .
RUN uv sync --frozen --no-dev

FROM python:3.14-slim
RUN useradd --create-home --uid 10001 app
WORKDIR /app
COPY --from=build --chown=app:app /app /app
ENV PATH="/app/.venv/bin:$PATH" PYTHONDONTWRITEBYTECODE=1 PYTHONUNBUFFERED=1
USER app
EXPOSE 8000
HEALTHCHECK CMD python -c "import urllib.request,sys; sys.exit(0 if urllib.request.urlopen('http://
127.0.0.1:8000/api/v1/readyz').status==200 else 1)"
CMD ["uvicorn", "app.main:create_app", "--factory", "--host", "0.0.0.0", "--port", "8000", "--workers",
"2", "--proxy-headers"]
```

Сайт и админка собираются в статику на этапе `node:24-alpine`, итоговые файлы копируются в образ Caddy (или монтируются в него томом). Отдельный сервер для фронтенда не нужен.

19.2. `compose.prod.yml` (суть)

```
services:
  db:
    image: postgres:18
    environment:
      POSTGRES_DB: app
      POSTGRES_USER: app
      POSTGRES_PASSWORD_FILE: /run/secrets/db_password
    volumes: [pgdata:/var/lib/postgresql]
    healthcheck: { test: ["CMD-SHELL", "pg_isready -U app"], interval: 5s, retries: 10 }
    restart: unless-stopped
    secrets: [db_password]

  migrate:
    image: app-api:${TAG}
    command: ["alembic", "upgrade", "head"]
    env_file: .env
    depends_on: { db: { condition: service_healthy } }
    restart: "no"

  api:
    image: app-api:${TAG}
    env_file: .env
    environment:
      FORWARDED_ALLOW_IPS: "*"      # порт api наружу не публикуется, доступ только у caddy
    depends_on: { migrate: { condition: service_completed_successfully } }
    restart: unless-stopped

  worker:
    image: app-api:${TAG}
    command: ["python", "-m", "app.worker"]
    env_file: .env
    depends_on: { migrate: { condition: service_completed_successfully } }
    restart: unless-stopped
```

```
  caddy:
    image: app-caddy:${TAG}       # Caddy + собранная статика сайта и админки
    ports: ["80:80", "443:443"]
    volumes: [caddy_data:/data, caddy_config:/config]
    depends_on: [api]
    restart: unless-stopped

  backup:
    image: postgres:18
    entrypoint: ["/backup/run.sh"]   # pg_dump каждые сутки, хранит 14 последних
    volumes: [./backup:/backup:ro, backups:/backups]
    env_file: .env
    depends_on: { db: { condition: service_healthy } }
    restart: unless-stopped

volumes: { pgdata: {}, caddy_data: {}, caddy_config: {}, backups: {} }
secrets: { db_password: { file: ./secrets/db_password } }
```

Внимание к тому PostgreSQL 18: в официальном образе `postgres:18` данные по умолчанию лежат в подкаталоге с номером версии внутри `/var/lib/postgresql`, поэтому том монтируется на `/var/lib/postgresql` (а не на `/var/lib/postgresql/data`, как было для старых версий). Перед деплоем исполнитель **MUST** сверить путь с документацией образа на Docker Hub для установленной версии.

Миграции выполняются **отдельным одноразовым сервисом** до старта API и воркера — не при старте приложения (чтобы два воркера uvicorn не запускали миграции одновременно).

19.3. Caddyfile

```
{
  email {$ACME_EMAIL}
}

(security_headers) {
  header {
    Strict-Transport-Security "max-age=31536000; includeSubDomains"
    X-Content-Type-Options "nosniff"
    Referrer-Policy "strict-origin-when-cross-origin"
    Permissions-Policy "camera=(), microphone=(), geolocation=()"
    -Server
  }
}

{$SITE_DOMAIN} {
  import security_headers
  encode zstd gzip
  handle /api/* {
    reverse_proxy api:8000
  }
  handle {
    root * /srv/site
    @assets path /assets/*
    header @assets Cache-Control "public, max-age=31536000, immutable"
    try_files {path} {path}/index.html
    file_server
  }
  handle_errors 404 {
    rewrite * /404.html
    file_server { root /srv/site }
  }
}

admin.{$SITE_DOMAIN} {
  import security_headers
  encode zstd gzip
```

```
  handle /api/* {
    reverse_proxy api:8000
  }
  handle {
    root * /srv/admin
    try_files {path} /index.html
    file_server
  }
}
```

CSP добавляется отдельными директивами для сайта и админки (раздел 15).

19.4. Бэкапы

- `pg_dump --format=custom` раз в сутки в том `backups`, имя `app-YYYY-MM-DD.dump`, хранятся 14 последних.
- **SHOULD:** дополнительно копировать дамп за пределы сервера (S3-совместимое хранилище российского провайдера).
- Проверка восстановления: `make restore-check` разворачивает последний дамп во временный контейнер и выполняет `SELECT count(*) FROM leads`. Запускается раз в неделю; бэкап, который не восстанавливается, бэкапом не считается.

20. Деплой на VDS Beget

Исполнитель выполняет по шагам и **отмечает выполнение в README**:

1. Узнать ОС на VDS (`cat /etc/os-release`). Рекомендуется Ubuntu LTS. Минимум ресурсов для проекта: 1 vCPU, 2 ГБ RAM, 20 ГБ диска.
2. Создать пользователя `deploy` с sudo; добавить SSH-ключ; в `sshd_config`: `PasswordAuthentication no`, `PermitRootLogin no`; перезапустить sshd **после** проверки входа по ключу в отдельной сессии.
3. `ufw allow 22,80,443/tcp && ufw enable`; установить `fail2ban`, `unattended-upgrades`.
4. Установить Docker Engine и Compose plugin по официальной инструкции Docker для этой ОС; добавить `deploy` в группу `docker`.
5. DNS: A-записи `example.ru` и `admin.example.ru` на IP VDS (в панели регистратора или Beget).
6. На сервере: каталог `/opt/app`, в нём `compose.prod.yml`, `.env` (права `600`), `secrets/db_password` (права `600`), `backup/run.sh`.
7. Образы: собрать в CI и отправить в реестр (GitHub Container Registry) **или** собрать на сервере (`docker compose build`) — для маленького проекта допустимо.
8. `docker compose -f compose.prod.yml up -d` → проверить `docker compose ps` (все healthy), открыть сайт, получить сертификат (Caddy делает это сам).
9. Создать администратора через CLI (раздел 7.7).
10. Отправить тестовую заявку → проверить письмо → войти в админку → увидеть заявку.
11. Проверить заголовки (`curl -I https://example.ru`) и пройти Lighthouse на проде.

Обновление: `TAG=<новый> docker compose -f compose.prod.yml up -d` — миграции выполнятся автоматически перед стартом API. **Откат:** вернуть предыдущий `TAG`. Миграции пишутся обратно совместимыми (сначала добавить колонку, потом использовать, потом удалить старую — в разных релизах), чтобы откат кода не ломал схему.

21. CI (GitHub Actions)

Файл `.github/workflows/ci.yml`, запускается на каждый push и pull request:

Задача	Шаги
`api`	setup uv → `uv sync` → ruff → mypy → pytest с покрытием (testcontainers использует Docker раннера) → pip-audit
`web`	setup pnpm + Node 24 → `pnpm install --frozen-lockfile` → lint → typecheck → vitest с покрытием → `i18n:check` → build → pnpm audit
`storybook`	build Storybook → test-runner с axe
`contract`	поднять API с БД → schemathesis по OpenAPI
`e2e`	`compose.test.yml up --wait` → Playwright → артефакты (трейсы, скриншоты) при падении
`lighthouse`	Lighthouse CI по собранному сайту с порогами из 12.6
`secrets`	gitleaks

Ветка `main` защищена: мердж только при зелёном CI.

22. План реализации

Каждый этап закрывается только при выполненном DoD и зелёных `make lint typecheck test`.

Этап	Содержание	Definition of Done
M0. Скелет	Монорепо, pnpm workspaces, uv-проект, общие конфиги, lefthook, Makefile, compose.dev.yml с db и mailpit, README с запуском	`make dev` поднимает пустые сервисы; линтеры и хуки работают
M1. Ядро API	settings, логирование, request id, problem+json, health, подключение к БД, Alembic, все таблицы раздела 6	Миграции up/down/up проходят; `/readyz` = 200; тест формата ошибок
M2. Приём заявок	Схемы, rate limit, honeypot, телефон, согласие, транзакция с outbox	Все тесты из 17.2 по заявкам зелёные
M3. Уведомления	Каналы email/log/max, воркер, backoff, фоновые задачи (очистка, обезличивание)	Письмо приходит в Mailpit; тесты воркера и обезличивания зелёные
M4. Авторизация	CLI create-admin, вход, сессии, CSRF, выход, смена пароля, отзыв сессий	Тесты авторизации из 17.2 зелёные
M5. API админки	Список с фильтрами, карточка, PATCH с версией, заметки, экспорт CSV; выгрузка OpenAPI	Тесты зелёные; schemathesis без ошибок; `make openapi` генерирует клиент
M6. Дизайн-система	Токены, тест контраста, i18n-пакет и `T()`, все компоненты раздела 10.3, `useForm`, `usePhoneMask`, Storybook	Каждое состояние каждого компонента в Storybook; axe без нарушений; покрытие ≥ 90%
M7. Сайт	Конфиг, все секции, форма, страницы, SEO, пререндер	E2E-сценарии 1–3, 9 зелёные; Lighthouse проходит пороги
M8. Админка	Вход, каркас, список, карточка, аккаунт	E2E-сценарии 4–8 зелёные
M9. Качество	CI целиком, axe на всех страницах, аудит зависимостей	CI зелёный полностью

Этап	Содержание	Definition of Done
M10. Прод	compose.prod, Caddy, образы, бэкапы, restore-check, деплой по разделу 20	Чек-лист раздела 23 выполнен на проде

23. Чек-лист приёмки

- ☐ Сайт открывается по HTTPS, HTTP редиректит на HTTPS.
- ☐ Заявка с телефона и с компьютера доходит; письмо приходит в течение минуты.
- ☐ Отключённый SMTP не ломает приём заявок; после восстановления письма доходят.
- ☐ Чекбокс согласия не отмечен по умолчанию; без него заявка не уходит.
- ☐ Ни один внешний запрос (шрифты, счётчики, карты) не уходит до действия пользователя — проверено во вкладке Network.
- ☐ В админке: вход, фильтры, поиск, карточка, статус, заметка, история, CSV, смена пароля, выход со всех устройств.
- ☐ Повторная попытка входа после 10 неудач блокируется.
- ☐ Lighthouse мобильный: Performance ≥ 95, Accessibility 100, Best Practices ≥ 95, SEO 100.
- ☐ Заголовки безопасности на месте (`curl -I`).
- ☐ Бэкап создаётся и восстанавливается (`make restore-check`).
- ☐ В логах нет персональных данных.
- ☐ В репозитории нет секретов (gitleaks чистый).
- ☐ README описывает: запуск dev, тесты, деплой, обновление, откат, создание и сброс администратора, смену контента под нового клиента.

24. Переиспользование в проектах 2 и 3

Проект строится так, чтобы следующие два проекта портфолио были дешёвыми:

- **Проект 2 — мини-приложение в MAX (запись/каталог).** Мини-приложения MAX работают на HTML, CSS и JavaScript, поэтому это ещё одно приложение `apps/max-app` на тех же `@app/ui`, `@app/i18n`, `@app/api-client`. API заявок уже есть (поле `source = 'max'` предусмотрено в схеме).
- **Проект 3 — ИИ-ассистент на сайт.** Новый модуль API `assistant` (база знаний бизнеса + вызов GigaChat или YandexGPT), виджет чата как компонент `@app/ui`. Если ассистент не может ответить — предлагает оставить заявку через ту же форму.

25. Типичные ошибки, которых нельзя допускать

1. Чекбокс согласия отмечен по умолчанию.
2. Токены и сессии в `localStorage`; JWT без возможности отзыва.
3. `CORS: *` вместо одного origin через прокси.
4. Тесты на SQLite вместо PostgreSQL.
5. Отправка письма синхронно в обработчике запроса (форма «зависает», заявка теряется при сбое SMTP).
6. Миграции при старте приложения в нескольких воркерах одновременно.

7. Открытые IP, телефоны и имена в логах.
8. Сырые строки в шаблонах и `v-html`.
9. `outline: none` без замены — невидимый фокус.
10. Контейнеры от root, порт базы данных открыт наружу.
11. Нет бэкапов или бэкапы ни разу не восстанавливались.
12. Выдуманные версии библиотек вместо установленных и зафиксированных в lock-файлах.
13. Выдуманные отзывы и «реальные клиенты» в демо.